

The Architecture of Intent — A Team Introduction

A 30-minute primer for teams adopting the framework alongside DevSquad Copilot

Marcel Aldecoa · *Independent practitioner*

Paper status: Team-intro variant (paper v0.1-team). ~3,500 words / ~8–10 pages. Written for engineers about to run their first Intent Design Session. The long-form paper is the arXiv reference; the workshop variant is the short academic submission; this version is the gateway you hand to a teammate the day before they start using the framework.

Framework version: v2.4.0 (2026-05-10). Aligned with the companion book.

Summary — the framework in 60 seconds

The Architecture of Intent is a working discipline for designing what AI agent systems are supposed to do, what they must never do, and how you will know they are working. It is opinionated, structural, and operationally complete: five activities to run (Frame · Specify · Delegate · Validate · Evolve), five archetypes to pick from, four orthogonal dials to calibrate, a 12-section spec template the agent executes against, a seven-category failure taxonomy that names which artifact to fix when something goes wrong, and four signal metrics that tell you whether the discipline is alive.

The cost of not having it shows up about a quarter into a serious agent deployment. The customer-support agent issues a \$0.00 refund because the spec said “process refund requests” without saying “amount must be > \$0.” The coding agent “fixes the failing test” by deleting it because the spec authorized “make the test pass” without forbidding “delete the test.” A platform team has fifty agents in production and twelve different teams have re-discovered the same prompt-injection vector against the same MCP server because no shared constraint library exists. Each incident is followed by a Slack-thread post-mortem that produces a one-line prompt patch and no structural change. By month six the prompts are war journals, the specs are decorative, and the discipline is gone.

For a DevSquad Copilot practitioner specifically, the framework adds five things DevSquad does not give you: an *archetype declaration in §4* of every spec; *NOT-authorized clauses in §3* (the single highest-leverage move); an *explicit oversight model* picked from four named options; the *seven-category fix-locus taxonomy* applied in retrospectives; and a *running Intent Design Session* for the next system. These five additions slot into the existing eight-phase cycle without inventing a parallel process. The Conductor agent reads the spec’s §4 archetype declaration to calibrate ceremony per phase. The specialist agents inherit constraints from the spec sections that pull them. The retrospective produces structural amendments instead of prompt patches.

Why a DevSquad team should care. DevSquad gives you a phase cadence, twelve specialist agents, a Conductor, MCP servers, and three impact tiers. What it does *not* give you is the *content* of the artifacts that move through the phases — the spec’s structure, the archetype’s calibration, the manifest’s boundary, the taxonomy that diagnoses failures. The framework fills that content gap. The framework and DevSquad are not alternatives; they are different layers of the same stack: **DevSquad is the *how*; the Architecture of Intent is the *what*.**

This applies to three deployment families DevSquad serves. DevSquad’s eight-phase cycle is not AI-specific. Teams use it to build (1) **new AI-agent systems** where the executor is an LLM; (2) **new cloud-native applications** — APIs, microservices, data pipelines, infrastructure-as-code modules — where the executor is deterministic code; and (3) **modernization and migration of existing applications**, where SDD becomes the discipline for moving a legacy app to a target architecture without regressing behavior. The framework’s central claim — *intent is a primary design surface for any delegated system* — is sharpest for AI agent systems because the executor is probabilistic, but the same discipline applies to the other two. An Executor archetype with a tool manifest and a NOT-authorized scope is just as load-bearing for a microservice that processes refunds as for an AI agent that does, and just as load-bearing for a legacy-modernization spec that says *what the modernized service must do, what it must never regress, and what counts as parity*. The four dials calibrate any of the three; the seven-category fix-locus taxonomy diagnoses any of them. Read this document with whichever deployment family you have in front of you; the framework slots in.

This document is the 30-minute primer. The companion book is the working manual.

What this is

You are about to start using the Architecture of Intent — probably because a teammate is running a pilot, because your team is co-adopting DevSquad Copilot and someone said the two compose well, or because you read the prologue of the book and want the rest of the team caught up before you sit down for the first Intent Design Session.

This document gives you exactly that. It is the framework in one read, written in the voice of the companion book rather than the voice of the academic paper. It does not argue for the framework; it teaches it. It does not develop novelty claims; it points to the chapters that do, if you want them later. It does not survey prior work; the long-form paper does. By the time you reach the end, you should be able to:

1. Recognize the five activities (Frame · Specify · Delegate · Validate · Evolve) and what each produces.
2. Pick an archetype for the system your team is about to build, and calibrate four dials independently.
3. Diagnose a wrong-output incident using the seven-category fix-locus taxonomy.
4. Read a spec written in the canonical 12-section template and tell whether it is ready to execute against.
5. Compose the framework with the DevSquad Copilot eight-phase cycle without inventing a parallel process.
6. Name what you will do in your first 30 days.

The book is the working manual. This document is the gateway. The two are aligned — every chapter the book teaches is referenced here by name.

Why this exists

Here is the failure pattern most teams adopting AI agents hit within a quarter.

The first demo is fast and impressive. The second pilot is slower than expected. By the third, the team is debugging output instead of shipping outcomes. Architectural coherence quietly degrades because no one is reviewing what the agent decided to do — only whether the test passed. The “AI made a mistake” incident reveals that nobody actually agreed, in writing, what the agent was authorized to do, what it must never do, or who was supposed to catch it when it drifted. Adding agents made the team faster on individual tasks and slower at shipping reliable systems. The bottleneck moved, but nobody renegotiated the work to match.

This is a structural problem, not a model problem. The model is doing what it was told. The trouble is that *being told* — what we ask of the agent, what we forbid it, how we check what it did — is the part the team did not learn how to do.

Senior engineers used to bridge this silently. They read the underspecified ticket, supplied missing constraints from experience, escalated ambiguity rather than executing it, and rewrote what was asked into what was clearly meant. The bridge worked because the implementer was also the judgment layer. When the implementer becomes an agent, the executor and the judgment layer separate. Agents have clarification APIs — Claude Code’s `AskUserQuestion`, Copilot’s interactive sessions, Cursor’s chat — but the *judgment* about when to invoke them is under-calibrated. The agent has the tool; the agent does not reliably use it. The cost of imprecise intent stops being absorbed silently and starts surfacing as wrong outputs, scope drift, and post-incident churn.

The Architecture of Intent is the discipline that makes intent thick enough to be governable when implementation is automated. Three load-bearing claims define it:

1. **Intent is a designed artifact**, distinct from implementation, requirements, and policy.
2. **Structural fixes compound; prompt fixes do not.** When something goes wrong, the fix lives in the spec, the manifest, a CI guard, or a framework version — not in the prompt alone.
3. **Calibration is deliberate.** Agency, autonomy, responsibility, and reversibility are four independent dials, not one “automation level.”

The rest of this document is what those three claims look like in practice.

What DevSquad gives you. What it doesn’t.

If you are reading this because you already use DevSquad Copilot and someone said “you should also adopt the Architecture of Intent,” the question worth being honest about is: *what does DevSquad already cover, and where does AoI extend it?* This section answers it for the three deployment families DevSquad serves — **new AI-agent systems**, **new cloud-native applications** (APIs, microservices, data pipelines, IaC modules), and **modernization or migration of existing applications using SDD**. The framework’s discipline applies to all three; this document is the gateway for whichever you are shipping.

DevSquad Copilot gives you, out of the box:

- An **eight-phase iterative cycle** for agentic software delivery (envisioning → spec → plan → decompose → implement → learn-in-the-open → review → refine). Phase boundaries are explicit; the Conductor enforces them.
- **Twelve specialist agents** (init, envision, kickoff, specify, plan, decompose, implement, review, security, sprint, refine, extend) plus a **Conductor** that orchestrates them. Each spe-

cialist has a defined surface; the Conductor decides which to invoke when.

- **Five first-party MCP servers** (GitHub, Azure DevOps, Azure, Microsoft Learn, Draw.io) covering the platform integration surface.
- **Three impact tiers** (low / medium / high) calibrating ceremony per change. Low skips most ceremony; high adds mandatory ADR, explicit approval, and full review.
- **Comprehension checkpoints** between phases. The Conductor verifies the team and the specialist agents understand the artifact before proceeding.
- The “**loop over ladder**” posture and a **20-skill catalog** with semantic activation.

This is a substantial substrate. A team running DevSquad has a working cadence, working agents, and a working tooling surface.

What DevSquad does *not* give you is the *content* of the artifacts that move through the phases:

- DevSquad gives you a *spec phase*; it does not give you the *spec template* with twelve named sections, an archetype declaration, a Composition Declaration, or a Cost Posture sub-block.
- DevSquad’s specialist agents will write a spec; the spec they write will not have an *archetype committed in §4* or *NOT-authorized clauses in §3* unless the team has committed to them as load-bearing.
- DevSquad’s three impact tiers calibrate *ceremony*; they do not calibrate *agency, autonomy, responsibility, or reversibility*. Two agents with the same impact tier may have very different blast radii.
- DevSquad’s *review phase* validates output. It does not, by itself, validate *intent* — the question of “is this the right thing to implement” runs upstream of code review.
- DevSquad’s *refine phase* re-prompts and re-runs. The framework adds the discipline that says *which artifact* the refinement amends — not the prompt alone.

The Architecture of Intent fills that content gap. It is the substantive vocabulary, calibration, and diagnostic structure that DevSquad’s process substrate runs on. The framework and DevSquad are not competitors; they are different layers of the same stack.

A note on the three deployment families. Most of this document uses AI-agent examples (a customer-support agent, a coding agent) because that is where the framework’s claims are most acute — the executor is probabilistic, the judgment gap is widest, the cost of imprecise intent is highest. But the framework does not require an AI executor to earn its keep:

- For **new cloud-native applications**, an Executor archetype with a tool manifest and a NOT-authorized scope is as load-bearing for a microservice that processes refunds as for an agent that does. The four dials calibrate a Kubernetes deployment pipeline as cleanly as they calibrate a coding agent. The seven-category fix-locus taxonomy diagnoses a wrong-output incident in a REST service the same way it diagnoses one in an LLM-driven agent.
- For **modernization and migration**, the spec is a parity contract: §1 (the legacy system being modernized) → §2 (the target outcome at parity) → §3 (in-scope vs. out-of-scope refactors; explicit migration boundary) → §4 (archetype of the modernized service; usually Executor or Synthesizer) → §6 (invariants that must not regress — security posture, public API contracts, data invariants) → §9 (acceptance criteria as parity tests) → §11 (per-step execution: the migration plan) → §12 (validation checklist: regression suites, performance budgets, on-call handoff). The framework’s discipline turns “rewrite this app” into a structured, reviewable, evolvable artifact rather than a multi-quarter open-ended migration project. Cat 1 failures (spec gaps) catch parity drift; Cat 6 failures (model-level — applied broadly: “the migrated implementation produced wrong output despite a correct spec”) catch genuine regressions

worth fixing in code.

When you see “agent” in this document, read “executor” — the framework’s claims hold whether you are shipping AI agents, cloud-native services, or modernized apps.

The framework on one page

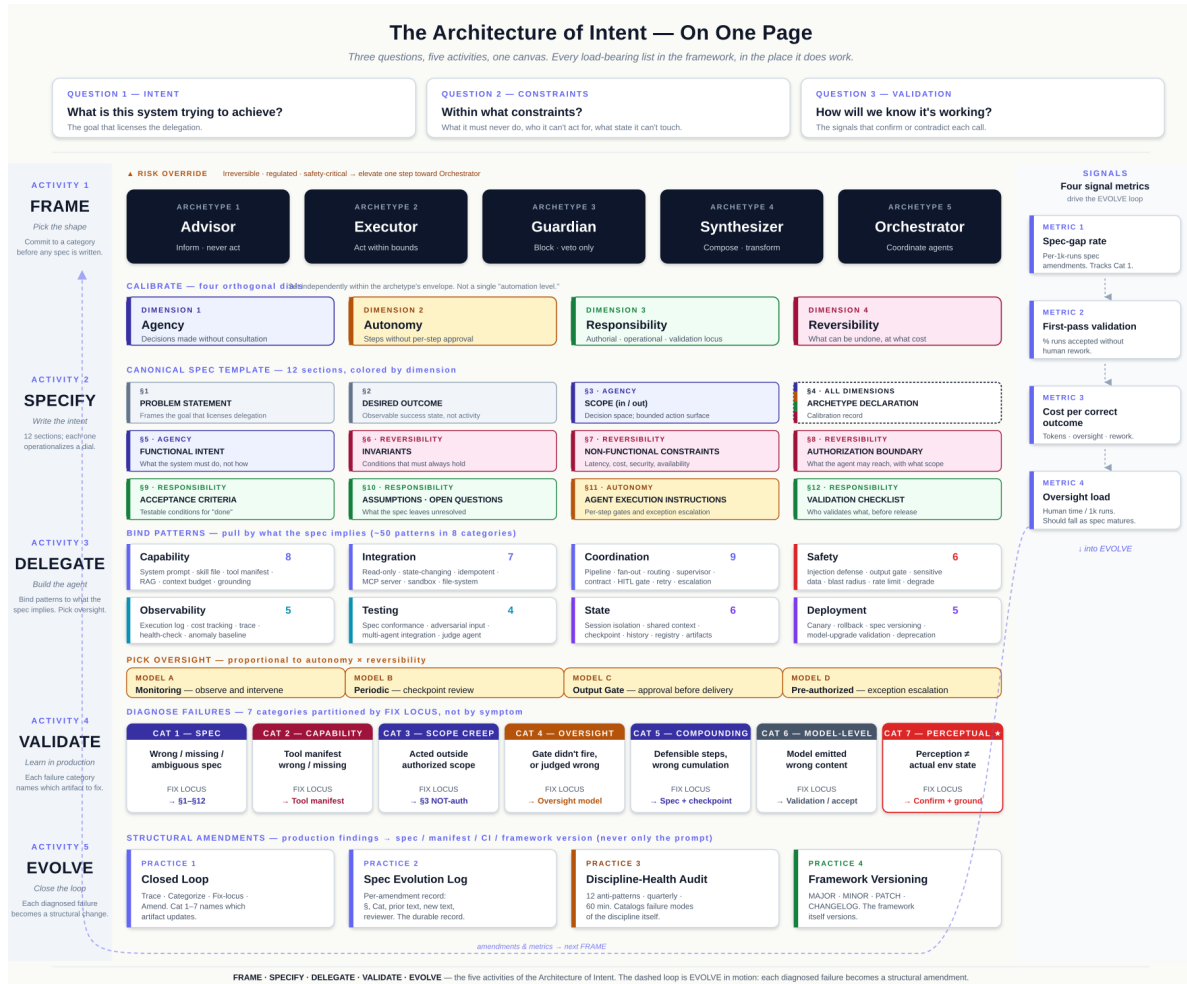


Figure 1: The Architecture of Intent on one page. Three questions every delegated system answers (top); the five activities that work them out (Frame · Specify · Delegate · Validate · Evolve, on the spine); the load-bearing constructs each activity binds; four signal metrics on the right rail that descend into the EVOLVE row, where each diagnosed failure becomes a structural amendment that feeds the next intent.

The canvas above is the framework in one picture. Every Part of the book elaborates one row. Every Intent Design Session walks left-to-right through the activities. When you get lost later, return to this picture.

The framework is organized around the answer to three questions every delegated system has to answer: *what is this system trying to achieve?* (intent), *within what constraints?* (the boundary),

how will we know it is working? (the signal). The five activities — Frame, Specify, Delegate, Validate, Evolve — work the three questions out. The rest of this document is a ~2-page summary per activity, plus the DevSquad Copilot integration story.

Frame — pick the shape

The first decision of any pilot is *what kind of system this is*. The framework’s working taxonomy is five canonical archetypes, each a pre-commitment to a behavioral envelope:

Archetype	What it does	Risk posture	Default oversight
Advisor	Surfaces information, options, recommendations; never acts on the world	Low	Human decides and acts
Executor	Carries out well-defined tasks within strict bounds	Medium	Pre-approved scope; exception-based escalation
Guardian	Enforces rules, validates integrity, blocks violations	Low (veto only)	Alerts; humans resolve
Synthesizer	Aggregates, distills, or composes from multiple sources into a coherent artifact	Medium	Output review above threshold
Orchestrator	Coordinates multiple agents or services toward a compound goal	High	Active oversight; escalation paths required

The selection tree resolves the choice in four sequential questions: does the system act on the world without a human between its output and the consequence? is its primary function protective rather than productive? does it coordinate other agents? is its primary product a synthesized artifact? Default after all four is *Executor*. A *risk override* elevates the archetype one step toward Orchestrator when actions touch irreversible state, regulated data, or safety-critical control.

Pre-commit before you design. A team that writes the spec first and then asks “is this an Advisor or an Executor?” is conflating intent with implementation. The pre-commitment makes downstream decisions traceable: the oversight model, the capability boundary, and the invariants all follow from the archetype.

Composition is first-class. A real deployment often hosts more than one archetype — an Orchestrator over Executors, with a Guardian gating each Executor’s output. Coding agents move between Synthesizer mode (planning) and Executor mode (writing files) within a session. The framework treats composition as a *first-class design surface* rather than as a sixth archetype; the Composing Archetypes chapter develops the Composition Declaration sub-block of §4 of the spec.

Once you have the archetype, you calibrate four dials — independently:

- **Agency** — the system’s decision space (what it is permitted to choose without consultation).
- **Autonomy** — the system’s execution path (how many steps run without per-step human approval).
- **Responsibility** — who is on the hook: authorial, operational, validation.
- **Reversibility** — what can be undone, at what cost.

These four are *orthogonal*. A compliance Guardian has wide agency (judges each transaction) but narrow autonomy (every flag surfaces for human resolution). A deterministic CI/CD pipeline has narrow agency (steps are fully prescribed) but wide autonomy (no per-step gates). Treating them as a single “automation level” collapses the design space onto a diagonal and costs you independent levers. Set each dial deliberately, in writing, in the spec — or each will be set for you by accident.

The Foundations chapters What is the Architecture of Intent and Calibrate Agency, Autonomy, Responsibility, Reversibility cover this in depth. Six chapter-scope diagrams in Part 0 make each concept visual.

Specify — write the artifact the agent executes against

The second activity is to turn the archetype and the four dials into an *executable* specification — a document the agent runs and a human validates against. The canonical template has twelve sections:

§	Section	Sets
1	Problem statement	The goal that licenses delegation
2	Desired outcome	Observable success state, not activity
3	Scope (in / out)	Agency — decision space
4	Archetype declaration + Composition Declaration + Cost Posture	All four dials; cost as a sub-block
5	Functional intent	What the system must do, not how
6	Invariants	Conditions that must always hold
7	Non-functional constraints	Latency, cost, security, availability
8	Authorization boundary	What the agent may reach, with what scope
9	Acceptance criteria	Testable conditions for “done”
10	Assumptions · open questions	What the spec leaves unresolved
11	Agent execution instructions	Per-step gates and exception escalation

Plus a *Spec Evolution Log* that records every amendment with the failure category that produced it.

The Living Spec discipline. Specs that never update describe systems no one is governing. Specs that update on every preference change are conversation transcripts. The middle path: *spec gaps* (Cat 1 below) and *spec ambiguities* trigger amendments; *implementation failures* (Cat 6) and *preference changes* do not. The Spec Evolution Log records the discipline.

The Intent Design Session. The framework’s working ritual is a 3–4 hour session — once per system, or once per major spec revision — that walks the team through seven phases: Frame the intent · Calibrate the dials · Draft the spec · Bind patterns · Pick oversight · Define acceptance · Plan the rollout. It produces the spec, the pattern set, the oversight model, the validation contract, and a deployment plan with a retrospective date on the calendar. Skipping it reliably produces five failure modes: patterns picked by team taste, oversight bolted on at the end, calibration left implicit, spec written by one person alone, no commitment to learning. The Intent Design Session chapter is the manual.

Design patterns for specs — what they actually buy you

If you are a hands-on engineer skeptical of “design patterns for specs,” you have probably read pattern-language books and seen the abstraction get more attention than the work. The framework’s pattern discipline is specifically *not* that. A “pattern” here is a named, structural slot in the spec template — not a piece of code, not a layer of architecture, not a theoretical abstraction. The patterns earn their keep by *changing what the agent does*, which is the only test that matters.

Here is what a spec looks like *without* the framework’s pattern discipline, written the way most teams write specs in 2026:

Process customer refund requests. Use the refund tool to issue the refund. Notify the customer when the refund is processed.

A DevSquad specialist agent will happily execute against this spec. The Conductor will route the work through the eight phases. The implement agent will write code. The review agent will check that the code compiles and the tests pass. The agent ships. Three days later it issues a \$0.00 refund to a customer who didn’t have an invoice for a cancelled order, because the spec said “process the refund” and did not say *what a refund is, when one is authorized, how much one can be, or what happens when the request is malformed*. The post-mortem reaches for the prompt: someone adds “make sure the refund amount is greater than zero” to the system prompt. Two weeks later a different shape of malformed refund happens. The prompt grows another sentence. By month three the prompt is a fragmentary war journal.

Here is the *same* spec with the framework’s design patterns applied — twelve named sections, each operationalizing a structural commitment:

§1 Problem. Customer-support team spends ~11 minutes per Tier-1 refund request. Automate the refund decision and issuance; surface a human only at the exceptions.

§2 Desired outcome. Refund requests with valid invoice, refund amount $\leq$ \$500, customer match, and order not cancelled produce a refund within 30s with audit trail and customer notification. All other shapes escalate to a human.

§3 Authorized scope. `invoice exists AND amount  $\leq$  $500 AND customer.id == order.customer_id AND order.status != "cancelled"`. **§3 NOT-authorized:** May not refund without invoice. May not refund amount $>$ \$500. May not refund cancelled orders. May not refund without customer-id match. May not issue \$0 or negative refunds.

§4 Archetype declaration. Executor (acts within bounded scope). Composition: none. **Cost Posture:** small-tier model $\cdot$ per-call ceiling \$0.02 $\cdot$ latency budget 5s. **Calibration:** agency narrow (single classification); autonomy wide within scope, narrow at boundary (escalates on any §3 NOT-authorized clause); responsibility distributed (support manager authorial; agent operational; same manager validation); reversibility moderate (refund reversible by re-charge within 7 days).

§6 Invariants. Refund amount $>$ \$0. Audit log within 5s. Customer notification within 30s. Spec evolution log updated on every Cat 1 amendment.

§8 Authorization boundary. Tool manifest: `read_invoice, read_order, read_customer, issue_refund` (state-changing), `notify_customer`. Specifically *not* exposed: `cancel_order, update_customer, delete_invoice`.

§11 Agent execution. If amount $>$ \$500 OR invoice missing OR cancelled OR customer mismatch $\rightarrow$ invoke `escalate_to_human` with the violated clause as reason. Do not retry. Log the escalation.

§12 Validation checklist. Pre-launch: 14 acceptance tests covering happy path + each NOT-authorized clause + escalation triggers. Production: spec-gap rate per 1k runs; first-pass validation rate; cost per correct outcome; oversight load.

This is what “design patterns for specs” means in the framework: each section is a *named slot* with a *structural purpose*, and every clause maps to either a behavioral commitment (the four dials) or an authorization boundary (the manifest). The framework’s Pattern Justification Map names which spec section pulls each pattern. A pattern that cannot be mapped to a section is inventory — either remove it or amend the template.

What changes about the agent’s behavior between the two specs above is concrete and immediate:

- The DevSquad implement agent now has explicit **NOT-authorized** constraints to encode in its execution path. It halts and escalates at the boundary instead of executing through it.
- The review specialist has a §12 checklist to audit against, not just “the code compiles.”
- The Conductor reads §4’s archetype and Cost Posture and picks the impact tier accordingly — small-tier model, \$0.02 per call ceiling, 5s latency.
- The refine specialist, when a failure surfaces, has a fix-locus partition (Cat 1–7) to consult; the amendment that lands is to *the named artifact* — not a prompt patch.

The patterns are not theoretical. They are the slots that tell the agents what to do.

Delegate — build the agent

The third activity binds the spec to the agent that will execute it. Three families of pattern matter here:

- **Capability patterns** — system prompt, skill file, tool manifest, per-task context, RAG, long-term memory, context-window budget, grounding. These tell the agent *what it knows*.
- **Integration patterns** — read-only tool, state-changing tool, idempotent tool, MCP server, function calling, code-execution sandbox, file-system access. These tell the agent *what it can reach*.
- **Coordination patterns** — sequential pipeline, parallel fan-out, conditional routing, supervisor, human-in-the-loop gate, retry with structured feedback, escalation chain. These tell the agent *how it composes with others*.

The framework’s discipline is that patterns are *picked by what the spec implies*, not by team enthusiasm. The Pattern Justification Map names which spec section pulls each pattern. A pattern that cannot be mapped to a spec section is inventory — either remove it or amend the template.

Oversight. Pick one of four models, proportional to the agent’s blast radius:

- **Monitoring** — observe and intervene; lowest friction, fits low-blast-radius work.
- **Periodic** — checkpoint review on a calendar.
- **Output Gate** — approval before delivery; fits Synthesizers and high-stakes Executors.
- **Pre-authorized** — agent operates within a budget without per-call approval; exceptions escalate. Fits well-tested Executors at scale.

The model is declared in §4 of the spec and operationalized in §11. The book’s Proportional Oversight chapter is the manual.

Validate — learn in production

When something goes wrong, the operationally useful question is not *what failed* but *which artifact must change so this doesn’t happen again*. The framework partitions failures into seven categories by *fix locus*:

Cat	Failure shape	Fix locus
1 — Spec	Wrong / missing / ambiguous spec clause	The spec
2 — Capability	Tool manifest wrong / missing / over-scoped	Tool manifest; capability authorization
3 — Scope creep	Agent acted outside authorized scope	Spec §3 NOT-authorized clauses
4 — Oversight	Gate didn’t fire, or judged wrong	Oversight model; gate configuration
5 — Compounding	Defensible steps, wrong cumulation	System-level invariant; checkpoint pattern
6 — Model-level	Model emitted wrong content despite correct spec/manifest/oversight	Structural validation; allowlist; accept residual risk

Cat	Failure shape	Fix locus
7 — Perceptual	Perceiving-then-acting system’s perception did not match actual environment state	Confirmation gate; screenshot-then-verify; multimodal grounding; element-allowlist

The first six synthesize common practice. Cat 7 — *Perceptual Failure* — is the framework’s novel category, for computer-use and browser-use agents whose failures include misidentification, missed element, hallucinated element, and state miscount. Prior taxonomies do not partition this class; the Computer-Use Agents chapter develops the structural fixes.

Four signal metrics close the loop: *spec-gap rate* (per-1k-runs spec amendments — tracks Cat 1), *first-pass validation* (% runs accepted without human rework), *cost per correct outcome* (tokens · oversight · rework), and *oversight load* (human time per 1k runs — should fall as the spec matures). These feed Evolve.

Evolve — close the loop

Each diagnosed failure produces a structural change. The Spec Evolution Log records what changed, why (which Cat), and who reviewed the amendment. The framework versions when load-bearing commitments move; everything else is local refinement.

The closed loop is the discipline that makes the practice compound. A team that fixes the prompt and moves on is running an *open* loop — the same failure shape recurs in the next system because the structural artifact (spec, manifest, CI guard) never absorbed the lesson. A team that fixes the structural artifact and amends the spec is running a *closed* loop: the next system inherits the lesson by inheriting the artifact.

The Closed Loop chapter is the manual. The Anti-Patterns catalog names the twelve ways the discipline decays — spec theater, oversight kabuki, metrics theater, pattern inventory, prompt-patch drift, archetype drift — and the Discipline-Health Audit is the quarterly cadence that catches them before they become terminal.

Operations — the sustaining layer

Part 6 of the book is the day-to-day machinery that keeps the discipline durable: proportional governance, cost and latency engineering, cacheable prompt architecture, production telemetry, the adoption playbook, multi-tenant fleet governance, and the DevSquad Copilot composition the next section walks through. It is *not* a sixth activity — the framework still commits to five — but it is the sustaining layer that runs alongside them. When the team scales from one agent to ten, this is the Part you live in.

Composing with DevSquad Copilot

Microsoft DevSquad Copilot, released in 2026, runs an eight-phase iterative cycle for agentic software delivery: *envisioning* → *spec* → *plan* → *decompose* → *implement* → *learn-in-the-open* → *review* → *refine*. Twelve named specialist agents plus a conductor run the phases. Five first-party MCP servers (GitHub, Azure DevOps, Azure, Microsoft Learn, Draw.io) cover the platform surface. Three impact tiers (low / medium / high) calibrate the ceremony.

The Architecture of Intent composes with this cycle cleanly:

DevSquad phase	Framework binding	What the framework adds
Envisioning	Frame · archetype selection	Pre-commit to one of five archetypes before the spec is drafted; risk override for high-consequence domains
Spec the next slice	Specify · the canonical 12-section template	Templated structure; archetype declaration in §4; NOT-authorized clauses in §3
Plan only what the current slice needs	Specify · Composition Declaration + Cost Posture sub-blocks	Cost commitment upstream rather than as a production-cost surprise
Decompose that slice	Delegate · pattern binding	Patterns picked by what the spec implies, not by team taste; Pattern Justification Map
Implement with TDD discipline	Delegate · tool manifest + system prompt + oversight model	The four oversight models, proportional to blast radius; capability-minimalist manifests
Learn in the open	Validate · four signal metrics + evals	Cat 1–7 categorization of every failure; metrics that drive Evolve
Review in an independent context	Validate · intent review before output review	Spec review as a practice; review the intent surface, not just the output
Refine continuously	Evolve · closed-loop discipline	Spec Evolution Log; Discipline-Health Audit; framework version bumps

The minimum additions that give a DevSquad team the most leverage, in priority order:

1. **Declare the archetype in §4 of every spec.** One line, but it constrains every downstream decision.
2. **Add NOT-authorized clauses to §3.** The single highest-leverage change. Most Cat 3 failures (scope creep) are catchable here.
3. **Pick one of the four oversight models explicitly.** Don't let “active oversight” mean “we'll figure it out.”

4. **Adopt the seven-category fix-locus taxonomy in retrospectives.** Each failure produces an amendment to the artifact the Cat names.
5. **Run an Intent Design Session for the next system.** Once. See what comes out. The chapter is the manual.

These five additions are the minimum viable Architecture of Intent for a team already running the DevSquad cycle. The Co-adoption with DevSquad Copilot chapter develops the full 30-day co-adoption plan and the vocabulary translation table.

How the Conductor agent uses the Architecture of Intent

The Conductor is the agent that orchestrates the eight phases — deciding which specialist to invoke, when to insert a comprehension checkpoint, when to escalate, when to retry. Without AoI, the Conductor has only DevSquad’s three impact tiers (low/medium/high) and the team’s informal sense of how careful to be. With AoI, the Conductor reads the spec’s §4 (archetype declaration + Composition Declaration + Cost Posture) on entry to each phase and uses it to calibrate every subsequent decision.

Concretely, phase by phase:

DevSquad phase	Without AoI	With AoI (what the Conductor reads, and what changes)
Envisioning	Team discusses the system informally; impact tier picked by gut.	Conductor walks the four-question archetype selection tree; selected archetype goes in §4 of the spec the <i>specify</i> phase will produce. First comprehension checkpoint asks the team to confirm the archetype.
Spec	<i>specify</i> agent writes a free-form spec.	Conductor invokes <i>specify</i> with the canonical 12-section template as directive. Verifies §3 (Scope) and §4 (Archetype) before allowing progression. Specs without explicit NOT-authorized clauses returned with “§3 <i>incomplete</i> .”
Plan	Plan written against the spec’s surface; cost considered if it surfaces.	Conductor reads §4’s Cost Posture sub-block. If the proposed plan exceeds the per-call cost ceiling or the latency budget, the phase halts and escalates <i>before</i> implement begins.

DevSquad phase	Without AoI	With AoI (what the Conductor reads, and what changes)
Decompose	Work split into specialist-agent units by the <i>decompose</i> agent.	Conductor reads §11 (Agent Execution Instructions) and decomposes accordingly. Each unit inherits the §6 (Invariants), §7 (Non-Functional Constraints), and §8 (Authorization Boundary) clauses relevant to its scope; the <i>decompose</i> specialist sees only the constraints relevant to its piece.
Implement	<i>implement</i> agent acts; impact tier shapes ceremony.	Conductor selects the implement agent variant by §4’s archetype: Executor → constraint-discipline variant; Synthesizer → composition variant; Orchestrator → supervisor-aware variant. §11 execution instructions become per-step rules. §8 tool manifest becomes the authorized capability set — no broader.
Learn-in-the-open	Telemetry per the team’s existing stack.	Conductor monitors the four signal metrics: spec-gap rate, first-pass validation, cost per correct outcome, oversight load. Drift flagged <i>before</i> it becomes a Cat 1 failure.
Review	Review specialist checks the code.	Conductor invokes review with a <i>two-axis</i> prompt: intent review (does this match §1–§4?) and output review (does this match §9 acceptance criteria?). Without AoI, both axes collapse into “does the code work.”

DevSquad phase	Without AoI	With AoI (what the Conductor reads, and what changes)
Refine	refine specialist re-prompts and re-runs.	Conductor walks the Cat 1–7 fix-locus taxonomy with the team. The amendment lands in <i>the artifact the Cat names</i> — spec for Cat 1, manifest for Cat 2, oversight model for Cat 4, etc. Prompt-level changes are the <i>last</i> resort, not the first.

The net effect: the Conductor stops being a routing layer and starts being a *governance layer*. The specialist agents inherit calibration from the spec rather than re-discovering it per task. The refine specialist’s amendments compound into the constraint library rather than fragmenting into prompt war journals.

Where DevSquad practice improves the most, in order

These are the *areas of greatest improvement* a DevSquad team sees after adopting AoI, ranked by leverage on a representative deployment:

1. **Cat 3 (scope creep) failures drop sharply.** The single highest-leverage move is making the §3 NOT-authorized clauses explicit. Most “the agent did something it shouldn’t” failures are catchable here.
2. **Refine amendments stop being prompt patches.** The Cat 1–7 fix-locus taxonomy gives the refine specialist a structured next-action. Specs and manifests evolve; prompts stop accumulating sentence fragments.
3. **The review phase becomes two-axis.** Intent review (review against §1–§4) becomes a distinct activity from output review. Most spec gaps are catchable here, *before* the agent has run against the spec.
4. **Cost surprises stop reaching production.** The Cost Posture sub-block of §4 commits the resource posture upstream; the Conductor enforces it at the plan phase. Cost-incident escalations move from production to design.
5. **Cross-system learning compounds.** The Spec Evolution Log and the Pattern Justification Map turn one system’s lesson into a fleet-wide constraint. The next agent on the platform inherits the lesson without re-discovering it.

Your first 30 days

If you have read this far and want to start, here is the rhythm the framework recommends.

Week 1. Pick *one* agent your team is about to build, has just shipped, or is about to revise. Bias toward “about to build” — the spec is most valuable before code exists. Read the Canonical Spec Template and write the spec for that one agent. Walk the Archetype Selection Tree. Declare the archetype in §4. Pick the oversight model. Ship the agent. What you are *not* doing in week 1: convincing other teams, writing process documents, building tooling, designing governance.

Month 1. The agent is running. Things will go wrong. Each failure: walk the diagnostic protocol from Failure Modes and How to Diagnose Them. Categorize. Identify the artifact to change. Each Cat 1 (Spec) updates the spec; the log entry records *which section was affected* and *why intent review didn't catch it*.

Quarter 1. A second team adopts, voluntarily, because they saw the first team's pilot ship cleanly. Demonstration beats mandate. Don't write a rollout plan; write a working example.

Year 1. The Spec Evolution Log and the constraint library are the team's most valuable assets. Engineers who joined after the framework was adopted have never written a prompt-only fix. The pattern compounds.

What to read next, by what you need

If you want to...	Read
See the whole framework in one screen before reading further	A Miniature Pilot, End-to-End
Write your first spec right now	The Canonical Spec Template
Pick the archetype for the agent you're designing	The Archetype Selection Tree
Diagnose a wrong-output incident	Failure Modes and How to Diagnose Them
Set up oversight for an agent about to ship	Proportional Oversight
Run the working ritual that produces a spec	The Intent Design Session
Build a coding agent specifically	Coding Agents + the Coding Agent worked pilot
Build a computer-use / browser-use agent	Computer-Use Agents (Cat 7 detail)
Adopt the framework across multiple teams	Adoption Playbook
Co-adopt with DevSquad Copilot	Co-adoption with DevSquad + DevSquad Mapping
Scale to many tenant teams sharing a platform	Multi-Tenant Fleet Governance
Check whether the discipline is still alive in a year	Signs Your Architecture of Intent Is Degrading
Submit the framework to a workshop	The workshop-length paper variant
Cite the framework academically	The long-form arXiv paper

What the framework is not

A few honest disclaimers, since reviewers reward narrow claims and overclaiming gets punished.

It is not a tutorial on a specific AI tool. It assumes you already have access to a capable model and tooling (Claude Code, Copilot, Cursor, Cline, or DevSquad's specialist agents). The discipline is the part the model and the tooling do *not* give you.

It is not a survey of the model landscape. Model selection is a Cost Posture decision in §4 of the spec, not a framework commitment. The book points at the right surface; it does not pick the model for you.

It is not a strategy document about whether to adopt AI. It assumes you have decided. It is the operational discipline that follows.

It does not promise that following these patterns guarantees a successful pilot. Models change, requirements shift, some failures are genuinely model-level and unfixable by better specs. What the framework gives you is the smallest set of structures that make a pilot’s failures *diagnosable* and *correctable* rather than mysterious. The structures compound; the leverage compounds.

One more page, then you’re done

If you have read this document end-to-end you have the framework. The five activities, the five archetypes, the four dimensions, the seven Cats, the four oversight models, the four signal metrics, the canonical 12-section spec template, the Intent Design Session, the closed loop, the composition story with DevSquad. ~3,500 words. That is the framework.

The book elaborates each of these elements across ~80,000 words of practitioner detail, three running scenarios threaded through every Part, ~50 patterns indexed against the spec template, and a handful of working rituals (the Intent Design Session, the Discipline-Health Audit, the Spec Evolution Log). You will reach for the book when something specific breaks or when you onboard the next teammate. This document is what you hand the teammate before that.

The bet the framework makes is that as delegation widens, intent precision compounds in value. A spec author whose hour produces a tightly-bounded specification commits the executor to hundreds of correct outputs; the same hour spent on a single output commits one. The leverage on intent has always been favorable; what 2024–2026 changes is that the leverage is now collectible.

Welcome to the team. We will see you at the next Intent Design Session.

Acknowledgments

The framework is in continuity with much standing work: GitHub spec-kit, Microsoft DevSquad Copilot, Anthropic *Building Effective Agents*, Shavit & Agarwal’s seven operational variables, SAE J3016’s six driving-automation levels, Cemri et al.’s MAST taxonomy, OWASP LLM Top 10. The long-form paper develops the lineage in §2 and Appendix A.